

IST604 — Web Services and Distributed Computing

First Semester Examination 2024/2025 — Model Answers

University of Buea, Faculty of Science — Department of Computer Science

Course: IST604 – Web Services and Distributed Computing

Instructor: Denis Nkweteyim

Date: 27/06/2025 | **Time:** 13:00–16:00 (3 hours)

Venue: U-BLOCK D | **Credit value:** 6

Instructions on paper: Answer all questions (70 marks total)

Question 1 — Web Services & SOAP/WSDL (30 marks)

(a) What is a web service? Explain three benefits (marks as allocated on paper)

Definition

A **web service** is a **standards-based, language-agnostic** software component that enables applications on **remote machines** to communicate over a network using **standard protocols** (such as HTTP) and **structured message formats** (such as XML or JSON). It accepts formatted requests and returns application-specific responses without requiring consumers to share the same platform or programming language.

Three benefits (any three from course — examples below)

1. **Loose coupling** — Client and server are independent; internal changes on one side do not break the other if the service interface (contract) remains the same.
2. **Ease of integration** — Web services connect isolated applications and data silos within or across organisations, acting as “glue” between heterogeneous systems.
3. **Service reuse** — A function is implemented once as a service and reused by many consumers over the network, reducing duplicate development and improving consistency.

(Other valid benefits from lectures: interoperability, scalability of integration, standards-based communication.)

(b) Interoperability, loose coupling, re-usability (brief explanations)

Advantage	Explanation
Interoperability	Different platforms and languages (e.g. Java client, .NET server) can communicate because messages use standard formats (XML/JSON) and standard protocols (HTTP).

Advantage	Explanation
Loose coupling	Consumer and provider do not depend on each other's internal implementation—only on the published interface (e.g. WSDL). Either side can evolve independently within that contract.
Re-usability	Business logic is exposed once as a service and invoked by many applications , avoiding repeated implementation of the same capability.

(c) RPC Style vs Document Style SOAP (difference)

Aspect	RPC Style	Document Style
Message structure	SOAP Body mimics a remote method call : wrapper named after the operation , with child elements per parameter .	SOAP Body contains a complete XML document as the message—not a direct parameter list.
Data types	Suited to simple types (String, int, etc.).	Supports complex/rich XML structures and extensible documents.
Industry use	Older/simpler style.	Industry standard and JAX-WS default (<code>@SOAPBinding</code> omitted → Document).
Annotation (JAX-WS)	<code>@SOAPBinding(style = SOAPBinding.Style.RPC)</code>	<code>@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)</code> or omit

Main difference: RPC maps operations to procedure-like calls with parameters; Document sends self-contained XML documents and is preferred for modern SOAP services.

(d) WSDL in SOAP-based web services

(i) Role of WSDL

WSDL (Web Services Description Language) is the **formal contract** between SOAP service provider and consumer. It is an **XML document** that describes:

- **What** operations the service offers,
- **What** data types/messages are used,
- **How** to bind to a protocol (e.g. SOAP over HTTP),
- **Where** the service endpoint is (URL).

Consumers use WSDL to generate clients (e.g. via `wsimport`) or to construct SOAP messages correctly. Providers publish WSDL (often at `?wsdl` on the endpoint URL).

(ii) Roles of WSDL elements

Element	Role
<types>	Defines data types used in messages, typically using XML Schema (XSD) . Describes structures of request/response payloads.
<binding>	Links an abstract portType/interface to a concrete protocol and message format (e.g. SOAP over HTTP, document/literal encoding).
<message>	Defines named data units exchanged—like parameters or parts of input/output for operations.
<portType> (WSDL 1.1) / <interface> (WSDL 2.0)	Groups abstract operations and their input/output messages —the logical interface of the service (what you can call).

(e) Role of the **wsgen** tool

wsgen is a JDK **command-line tool** used in **bottom-up (code-first)** SOAP development by the **service provider**.

Role:

- Takes a compiled class annotated with **@WebService** and **generates supporting artifacts** (e.g. JAXB classes for XML binding).
- With the **-wsdl** flag, it can produce a **physical WSDL file** from the implementation before or after deployment.
- Helps share the service contract with client developers who will use **wsimport** to build clients.

Note: For simple JAX-WS apps, **Endpoint.publish()** may generate equivalent runtime metadata dynamically; **wsgen** is used when you need an explicit WSDL file to distribute.

Question 2 — JAX-WS TimeServer2 Example (20 marks)

(a)(i) What each class represents

Concept	Meaning in JAX-WS practice
Interface (SEI)	Service Endpoint Interface — declares the web service contract (operations exposed to clients).
Service Implementation Bean (SIB)	Class that implements the SEI — contains the business logic for each @WebMethod .
Publisher / server class	Class with main that deploys the service using Endpoint.publish() .
Client	Application that consumes the service via WSDL + Service.create() + getPort() .

(a)(ii) Map code fragments to roles

Fragment	Lines	Represents
Service Implementation Bean (SIB)	1–8	<code>TimeServer2Implementation</code> — implements the interface and provides <code>getTimeAsString()</code> / <code>getTimeAsElapsed()</code> .
Interface (SEI)	9–17	<code>TimeServer2Interface</code> — <code>@WebService</code> , <code>@WebMethod</code> , <code>@SOAPBinding(RPC)</code> .
Publisher (web service host)	18–23	<code>TimeServer2</code> — <code>Endpoint.publish(...)</code> starts the HTTP endpoint.
Client	24–33	<code>TimeClient</code> — loads WSDL, creates <code>Service</code> , gets port, invokes methods.

(b) Two parameters on Line 20

```
Endpoint.publish("http://127.0.0.1:9873/ts", new TimeServer2Implementation());
```

Parameter	Meaning
1st — <code>"http://127.0.0.1:9873/ts"</code>	Publication URL — address where the service listens; WSDL is typically at <code>http://127.0.0.1:9873/ts?wsdl</code> .
2nd — new <code>TimeServer2Implementation()</code>	Instance of the SIB — object that handles incoming SOAP requests and executes the web methods.

(c) Two parameters on Line 28 (QName constructor)

```
QName qname = new QName("http://jws.ist60420242025/",  
"TimeServer2ImplementationService");
```

Parameter	Meaning
1st — <code>"http://jws.ist60420242025/"</code>	Namespace URI — must match the target namespace of the service in the WSDL.
2nd — <code>"TimeServer2ImplementationService"</code>	Local name / service name — identifies the <code><service></code> element in the WSDL (used by <code>Service.create(url, qname)</code>).

(Line 30 also uses `url` and `qname` in `Service.create(url, qname)` — URL points to WSDL; QName identifies which service definition to use.)

(d) Information at the endpoint in the browser

When you open the published endpoint (or `?wsdl`), JAX-WS exposes service metadata. Typical labels:

Label on page	What you find (for this example)
Service Name	<code>TimeServer2ImplementationService</code>
Port Name	Often <code>TimeServer2ImplementationPort</code> (or similar generated port name)
Address	<code>http://127.0.0.1:9873/ts</code> (endpoint URL)
WSDL	Link/URI to WSDL document: <code>http://127.0.0.1:9873/ts?wsdl</code>

(Exact port name follows JAX-WS naming conventions from the implementation class; exam answers should match what appears on the generated page.)

Question 3 — REST & HTTP Client (20 marks)

(a) Four REST verbs and CRUD (6 marks)

HTTP Verb	CRUD operation	Brief explanation
GET	Read	Retrieve a resource without changing server state (e.g. fetch a web page or record).
POST	Create	Submit data to create a new resource or trigger processing (e.g. new record, form submission).
PUT	Update (full replace)	Replace an existing resource with the representation sent in the body.
DELETE	Delete	Remove a resource identified by the URL.

Note: REST is resource-oriented; CRUD mapping is the standard teaching alignment used in the course.

(b) `SimpleHttpClient` with `http://www.ubuea.cm/index.html`

(i) Values of `host` and `path` (Lines 9–10)

Variable	Value
<code>host</code> (line 9)	<code>www.ubuea.cm</code>
<code>path</code> (line 10)	<code>/index.html</code>

(Port: `url.getPort()` is `-1` when omitted, so line 12–14 sets `port = 80` for HTTP.)

(ii) Value of `request` after Line 15 and after Line 16

After Line 15 executes:

```
GET /index.html HTTP/1.1
```

After Line 16 executes (Line 17 adds the blank line that ends headers):

```
GET /index.html HTTP/1.1
host: www.ubuea.cm
```

(Line 17: `request += "\n\n";` — blank line after headers, required before HTTP body.)

(iii) What `sock` represents; Lines 19–27

`sock` (Line 18):

A `java.net.Socket` object — a **TCP connection** to the web server at `host` (`www.ubuea.cm`) on `port` (80).

What happens between Lines 19 and 27:

1. **Line 19–21:** A `PrintWriter` sends the HTTP **request string** over the socket to the server
(`writer.print(request); writer.flush();`).
2. **Lines 22–23:** A `BufferedReader` reads the server's **HTTP response** from the socket input stream.
3. **Lines 24–27:** The client reads the response **line by line** and prints each line to the console (status line, headers, then body).
4. **Line 28:** The socket is **closed**, ending the connection.

Summary: The program manually implements a minimal **HTTP GET client**: connect → send request → read response → close.

Mark distribution summary (from paper)

Question	Marks
Q1	30
Q2	20
Q3	20
Total	70

End of model answers — First Semester Exam, 27 June 2025.